

LangChain Agent

Agentic AI Python Tutor — Sprint 2



LLM wünscht — LangChain erfüllt



AGENT



LLM



TOOLS



RAG + SEITEN

AGENDA

Was haben wir **gebaut**?

01 — INTERFACE

UI zur Interaktion

Next.js Frontend — Student schreibt Code, stellt Fragen, lädt PDF hoch und sieht sofort die Antwort des Agents.



02 — BACKEND

FastAPI Endpoint

Empfängt Requests vom Frontend und leitet sie an den Agent weiter. Endpunkte: */analyze*, */chat*, */run*, */upload*



03 — KERN

LangChain Agent

Orchestriert den Tool-Loop — LLM entscheidet selbst welche Tools aufgerufen werden bis die Antwort fertig ist.



04 — TOOLS

Spezialisierte Tools

4 Tools: *explain* · *debug* · *exercise* · *rag* — jedes mit eigenem LLM-Prompt.



05 — LLM

OpenAI · GPT

GPT-4o über die OpenAI API (Standard) — Ollama/llama3.2 als lokaler Fallback wenn kein API Key gesetzt.



06 — WISSEN

RAG Pipeline

PDF → Chunks mit Seitennummer → FAISS Vector Store. Seitenreferenz ("Seite 9") direkt abrufbar + semantische Suche kombiniert.

FRONTEND — KURZ

Was der Student sieht und macht

**Code-Editor**

Python-Code schreiben und direkt im Browser ausführen

**Code analysieren**

Ein Klick — Agent erklärt, findet Fehler, gibt Übung

**Chat mit dem Tutor**

Fragen stellen mit vollem Code-Kontext — nur Python-Fragen erlaubt

**PDF hochladen**

Eigenes Kursmaterial einbinden — Agent nutzt es bei Antworten

**Code ausführen**

Python-Code wird sicher im Backend ausgeführt, Output sofort angezeigt

**„In Editor“ Button**

Python-Code aus Chat-Antworten per Klick in den Editor übernehmen und sofort ausführen

**Material-Badge im Header**

Nach PDF-Upload zeigt der Chat-Header den Dateinamen — sichtbar dass Lernmaterial aktiv ist

• Python Tutor



Hallo! Ich bin dein Python-Tutor.

Stell eine Frage, analysiere deinen Code oder lade ein PDF hoch.

📎 Frage stellen... (Enter = senden)

PDF anhängen für kontextbasierte Analyse

BACKEND

FastAPI — warum ideal für KI-Agents

API ENDPUNKTE — /TUTOR/*

POST	/tutor/analyze	Code → Agent → 5 Felder zurück
POST	/tutor/chat	Frage + Code + Verlauf → Antwort
POST	/tutor/run	Code ausführen → stdout / stderr
POST	/tutor/upload-material	PDF → Chunks → FAISS Vector Store

VORTEILE FASTAPI FÜR KI-AGENTS

⚡ Async — kein Blockieren

- `upload-material` ist `async def` — PDF-Upload blockiert den Server nicht
- Chat, Analyze und Run sind `sync def` — LLM-Calls laufen direkt durch

🛡 Pydantic Validierung

- Automatische Typprüfung bei jedem Request
- Fehler werden sofort erkannt — kein falscher Input zum Agent

🎯 Klare Trennung

- Jeder Endpoint hat eine Aufgabe — kein gemischter Code
- Agent-Logik bleibt im Backend — Frontend sendet nur Daten

🔒 Sicherheit

- Code-Ausführung isoliert mit Timeout (10 Sekunden)
- Off-Topic Classifier blockt irrelevante Anfragen
- Regex erkennt „Seite 9“-Referenzen → direkte Seitensuche statt FAISS

KERN DER ARCHITEKTUR

LangChain Agent — wie er denkt

⚙️ **_SYSTEM_PROMPT**

Steuert Sprache, Tonalität und Antwortformat — definiert das Verhalten des Agents

📦 **Was sind tool_calls?**Das LLM antwortet nicht mit Text — sondern mit einem **JSON-Objekt**:

```

{
  "tool_calls": [{
    "name": "explain_code_tool",
    "args": { "code": "..." }
  }]
}

```

LangChain liest dieses Feld und führt das Tool aus. Ist `tool_calls` leer → Loop stoppt.

🤖 LLM WÜNSCHT SICH — ⚙️ LANGCHAIN ERFÜLLT

**LLM — Wunsch (tool_calls)**Ich brauche `explain_code_tool` und `debug_code_tool` für diesen Code.**LangChain — führt aus**

Tools werden aufgerufen... Ergebnis: „Code gibt 0-4 aus. Kein Fehler.“

**LLM — nächster Wunsch (tool_calls)**Danke. Jetzt brauche ich noch `exercise_tool`.**LangChain — führt aus**

Übung generiert: „Schreibe eine Schleife von 1-10.“

**LLM — fertig, tool_calls = [] → Loop stoppt****Erklärung:** Schleife 0-4 | **Fehler:** nein | **Übung:** Schleife 1-10

AGENT TOOLS

Die 4 Tools des Agents

**explain_code_tool**`agent/tools/explain_tool.py`

Erklärt den Code auf Deutsch in 3–4 Sätzen. Was macht er, wie funktioniert er, einfache Sprache für Anfänger.

ERKLÄRUNG

**debug_code_tool**`agent/tools/debug_tool.py`

Analysiert den Code auf Syntax- und Logikfehler. Gibt Fehlertyp und konkreten Verbesserungsvorschlag zurück.

FEHLER FINDEN

**exercise_tool**`agent/tools/exercise_tool.py`

Generiert eine passende Übungsaufgabe basierend auf dem Code des Studenten — maximal 3 Sätze, direkt anwendbar.

ÜBUNG

**rag_tool**`agent/tools/rag_tool.py`

Sucht im FAISS Vector Store nach relevanten Stellen. Im /analyze-Agent verwendet. Im /chat läuft `_get_rag_context()` direkt: erst Seitensuche, dann semantische FAISS-Suche — Ergebnisse kombiniert.

OPTIONAL



Wie das LLM das richtige Tool wählt: Jedes Tool hat einen Docstring — dieser Docstring ist die Beschreibung die das LLM bekommt. Das LLM liest ihn und entscheidet selbst. **Guter Docstring = korrektes Tool.**

RAG — RETRIEVAL AUGMENTED GENERATION

PDF → Wissen für den Agent

UPLOAD-PIPELINE

**1 • PDF hochladen & Text extrahieren**

POST /tutor/upload-material → `extract_pages()` gibt **(Seitennummer, Text)** Tupel pro Seite zurück

**2 • Splitten & Embeddings**

`split_pages()` → **(Chunk-Text, Seitennummer)** · Jeder Chunk wird als Vektor eingebettet

**3 • Speichern & Suchen**

FAISS Vector Store lokal · Bei Anfrage: Seitenzahl erkannt → `get_page()` + FAISS-Semantiksuche kombiniert

**Was ist RAG & wie funktioniert Ähnlichkeitssuche?**

Das LLM sucht zuerst im eigenen Material nach relevanten Stellen. Fragen und Chunks werden als Vektoren gespeichert — FAISS findet die thematisch nächsten in Millisekunden.

**Optional • GPT-4o Standard • Ollama Fallback**

`rag_tool` nur aktiv wenn FAISS-Index vorhanden. Standard: OpenAI GPT-4o. Fallback: Ollama/llama3.2 lokal wenn kein API Key gesetzt.

**Seitenreferenz + Semantik kombiniert**

„Erkläre Seite 9“ → Regex extrahiert Seitennummer → `get_page()` direkt + FAISS-Suche parallel — Ergebnisse zusammengeführt, Duplikate entfernt.

WARUM KEIN CHATBOT?

Agent vs. ChatGPT / Chatbot

08 / 09



ChatGPT / Chatbot

Reagiert auf Text — antwortet mit Text

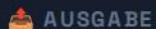
🧠 VERARBEITUNG

LLM generiert Antwort **direkt** — kein Werkzeug, keine Zwischenschritte



LOOP

Keiner — eine Frage, eine Antwort, fertig



AUSGABE

Ein freier Textblock — keine feste Struktur, keine spezialisierten Aktionen



Unser LangChain Agent

Plant, handelt, überprüft — eigenständig

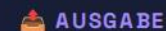
🧠 VERARBEITUNG

LLM entscheidet welche **Tools** gebraucht werden — explain · debug · exercise · rag



LOOP

Läuft bis `tool_calls` leer ist — mehrere Runden möglich



AUSGABE

5 strukturierte Felder: Erklärung · Fehler gefunden · Fehlertyp · Hinweis · Übung

VS

Live Demo